

BẢN GIẢI THÍCH CHI TIẾT CHƯƠNG TRÌNH PYTHON (CODE CORE)

Đề tài: Phát hiện giao dịch gian lận bằng thuật toán Isolation Forest

1. Khai báo các thư viện hỗ trợ

```
D: > python > baitapnhom.py
1  import pandas as pd
2  import numpy as np
3  from sklearn.ensemble import IsolationForest
4  from sklearn.preprocessing import StandardScaler, OneHotEncoder
5  from sklearn.compose import ColumnTransformer
6  #
7  #
8  #
```

- **pandas (pd):** Thư viện dùng để đọc, thao tác và quản lý dữ liệu dạng bảng (Dataframe) từ file CSV.
- **numpy (np):** Thư viện tính toán đại số tuyến tính, dùng để xử lý mảng và chuyển đổi giá trị nhân dự đoán.
- **IsolationForest:** Thuật toán Machine Learning chính (thuộc nhóm Học không giám sát - Unsupervised Learning) dùng để phát hiện các điểm dữ liệu dị biệt/bất thường.
- **StandardScaler:** Bộ bộ chuẩn hóa dữ liệu dạng số theo phân phối chuẩn (Z-score normalization).
- **OneHotEncoder:** Bộ chuyển đổi các biến dữ liệu dạng chữ/danh mục (Categorical) thành chuỗi mã hóa nhị phân 0 và 1.
- **ColumnTransformer:** Công cụ gom nhóm và áp dụng đồng thời các kĩ thuật xử lý khác nhau cho từng tập cột dữ liệu khác nhau.

2. Đọc dữ liệu và Phân loại 26 đặc trưng (Feature Partitioning)

```
# 1. Đọc file CSV từ Kaggle
df = pd.read_csv('filemaudulieu.csv') # Thay tên file CSV thực tế của bạn vào đây

# =====
# 2. CHIA CỘT RÕ RÀNG (KHỚP 100% VỚI 26 CỘT CỦA BẠN)
# =====

# Nhóm 1: Các cột dạng SỐ (Chạy qua StandardScaler) - 13 cột
features_numeric = [
    'amount_usd',           # Số tiền giao dịch
    'hours_since_last_txn', # Số giờ từ giao dịch trước
    'txn_count_last_24h',   # Số giao dịch trong 24h qua
    'distance_from_home_km', # Khoảng cách từ nhà (km)
    'card_age_months',      # Tuổi đời của thẻ (tháng)
    'customer_age',         # Tuổi khách hàng
    'account_balance_usd',  # Số dư tài khoản
    'cvv_retry_count',      # Số lần gõ sai mã CVV
    'velocity_score',       # Điểm tốc độ giao dịch
    'time_of_day_hour',     # Giờ trong ngày (0 - 23)
    'day_of_week',          # Ngày trong tuần (0 - 6)
    'merchant_risk_score',  # Điểm rủi ro của cửa hàng
    'prior_disputes'        # Số lần tranh chấp/khiếu nại trước đó
]
```

- **pd.read_csv():** Đọc file dữ liệu đầu vào chứa danh sách các giao dịch thẻ tín dụng và lưu dưới dạng biến df.
- **Ý nghĩa:** Tập hợp **13 cột dữ liệu liên tục (Số)**.
- **Lý do tách nhóm:** Các thông số như số tiền (amount_usd), tuổi thẻ, khoảng cách... có giá trị số học và khoảng chênh lệch khác nhau. Mô hình Machine Learning yêu cầu các thông số này phải được đưa về cùng một khoảng giá trị (scale) để tránh việc biến có số đo lớn áp đảo biến có số đo nhỏ.

```
features_categorical = [
    'merchant_category',    # Ngành hàng cửa hàng
    'card_type',            # Loại thẻ
    'auth_method',          # Phương thức xác thực
    'channel',              # Kênh giao dịch
    'device_type',          # Loại thiết bị
    'is_foreign_transaction', # Giao dịch nước ngoài (True/False)
    'is_new_merchant',      # Cửa hàng mới giao dịch lần đầu
    'used_vpn',             # Có dùng VPN hay không
    'ip_country_mismatch',  # IP khác quốc gia đăng ký
    'billing_shipping_mismatch', # Địa chỉ thanh toán khác địa chỉ giao
    'is_ai_generated_scam_attempt' # Dấu hiệu lừa đảo bằng AI
]
```

- **Ý nghĩa:** Tập hợp **11 cột dữ liệu phân loại (Chữ / Cờ True-False)**.
- **Lý do tách nhóm:** Máy tính không thể trực tiếp tính toán đại số trên chuỗi ký tự (như "web", "mobile", "Visa"). Do đó, các biến này bắt buộc phải qua bước mã hóa thành các con số 0 và 1.

```
# Nhóm 3: Các cột BỎ QUA (2 cột)
# - 'transaction_id': Mã định danh ngẫu nhiên (không có giá trị học)
# - 'is_fraud': Nhãn kết quả thực tế (chỉ dùng để kiểm tra, không đưa vào mô hình Unsupervised)

all_features = features_numeric + features_categorical
```

- **transaction_id:** Mã định danh duy nhất của từng giao dịch (chỉ là chuỗi ngẫu nhiên, không chứa quy luật học) nên bỏ ra.
- **is_fraud:** Nhãn thực tế. Vì đây là bài toán Học không giám sát (Unsupervised) dùng IsolationForest, mô hình sẽ tự tìm ra giao dịch bất thường dựa trên hình thái dữ liệu chứ không học từ đáp án sẵn có, nên ta loại cột này ra khỏi quá trình huấn luyện.
- **all_features:** Gồm tổng cộng 24 đặc trưng cần thiết để đưa vào huấn luyện.

3. Tiền xử lý dữ liệu (Data Preprocessing Pipeline)

A. Xử lý ô trống (Handling Missing Values)

```
5
7 # Điền ô trống (Missing Values) để tránh lỗi crash code
8 for col in features_numeric:
9     if col in df.columns:
10         df[col] = df[col].fillna(df[col].median())
11
12 for col in features_categorical:
13     if col in df.columns:
14         df[col] = df[col].fillna(df[col].mode()[0])
15
```

- **df[col].fillna(...):** Lấp đầy các ô trống (giá trị thiếu/NaN) trong file dữ liệu để tránh làm suy giảm chất lượng hoặc gây lỗi gián đoạn chương trình khi chạy mô hình.
- **df[col].median():** Với biến số, điền ô trống bằng **Trung vị** (Median) thay vì Trung bình (Mean) để không bị ảnh hưởng bởi các giá trị ngoại lệ quá lớn.
- **df[col].mode()[0]:** Với biến chữ/True-False, điền ô trống bằng **Yếu vị** (Mode - giá trị xuất hiện nhiều nhất).

B. Chuẩn hóa & Mã hóa biến dữ liệu

```
# Dùng ColumnTransformer để biến đổi dữ liệu
preprocessor = ColumnTransformer(
    transformers=[
        ('num', StandardScaler(), features_numeric),
        ('cat', OneHotEncoder(handle_unknown='ignore', sparse_output=False), features_categorical)
    ]
)
```

- **StandardScaler()**: để chuyển đổi toàn bộ 13 cột số về cùng 1 quy mô(thang đo) để mô hình tính toán công bằng, không bị lệch
- **OneHotEncoder(handle_unknown='ignore', sparse_output=False)**:
 - dùng để **chuyển các cột dạng chữ/danh mục thành chuỗi số nhị phân 0 và 1** để máy tính đọc được.
 - handle_unknown='ignore': Bỏ qua và gán giá trị 0 nếu gặp một danh mục lạ chưa xuất hiện lúc huấn luyện.
 - sparse_output=False: Trả về mảng dữ liệu đầy đủ thay vì ma trận thưa (Sparse Matrix) để tương thích dữ liệu.

```
# Chuyển đổi toàn bộ 24 cột tính năng thành mảng số X_encoded
X_encoded = preprocessor.fit_transform(df[all_features])

# =====
# 4. HUẤN LUYỆN MÔ HÌNH ISOLATION FOREST
```

- **fit_transform(...)**: Thực hiện 2 công việc liên tiếp:
 1. fit: Học các thông số (giá trị trung bình, độ lệch chuẩn, các danh mục duy nhất) từ tập dữ liệu df[all_features].
 2. transform: Biến đổi toàn bộ dữ liệu thô sang dạng ma trận số chuẩn hóa hoàn chỉnh và lưu vào biến X_encoded.

4. Khởi tạo và Huấn luyện mô hình Isolation Forest

```

model = IsolationForest(
    contamination=0.015, # Dự đoán 1.5% giao dịch bất thường
    n_estimators=100,    # Khởi tạo 100 cây cô lập
    random_state=42,     # Cố định ngẫu nhiên
    n_jobs=-1            # Tận dụng tối đa nhân CPU
)
model.fit(X_encoded)

```

- **IsolationForest(...)**: Khởi tạo thuật toán cây cô lập dựa trên nguyên lý: "Các điểm dữ liệu bất thường rất ít và khác biệt, nên chúng dễ bị cô lập chỉ sau một vài lần chia nhánh ngẫu nhiên trên cây quyết định".
- **Cấu hình tham số cốt lõi**:
 - **contamination=0.015**: Tỷ lệ nhiễm/bất thường dự kiến. Thiết lập 0.015 đồng nghĩa giả định có khoảng 1.5% số giao dịch trong dữ liệu là gian lận/bất thường.
 - **n_estimators=100**: Xây dựng rừng gồm 100 cây cô lập ngẫu nhiên để tăng độ chính xác và ổn định.
 - **random_state=42**: Đặt điểm khởi tạo ngẫu nhiên cố định để đảm bảo mỗi lần nhóm chạy lại code đều thu được đúng một kết quả giống nhau.
 - **n_jobs=-1**: Cho phép chương trình huy động 100% công suất các nhân CPU trên máy tính nhằm tăng tốc độ xử lý.
- **model.fit()**: Đưa toàn bộ ma trận dữ liệu đã tiền xử lý X_encoded vào để mô hình xây dựng 100 cây quyết định cô lập.

5. Gán nhãn dự đoán và Xuất báo cáo (Output/Export)

```

preds = model.predict(X_encoded)
df['is_anomaly'] = np.where(preds == -1, True, False)

```

- **model.predict(X_encoded)**: Hàm trả về một mảng chứa hai giá trị dự đoán:
 - **1**: Giao dịch bình thường (nằm sâu bên trong phân phối dữ liệu).
 - **-1**: Giao dịch bất thường/nghi ngờ gian lận (bị cô lập nhanh chóng).
- **np.where(preds == -1, True, False)**: Chuyển đổi mã nhãn kỹ thuật (-1 và 1) thành cờ Boolean trực quan (True - Có bất thường, False - Bình thường) và ghi thành cột mới is_anomaly vào dataframe.

```
# In ra 10 giao dịch bất thường đầu tiên
anomalies = df[df['is_anomaly'] == True]
output_filename = 'danh_sach_bat_thuong_day_du.csv'
anomalies.to_csv(output_filename, index=False, encoding='utf-8-sig')
```

- **anomalies = df[...]**: Lọc chỉ lấy các dòng dữ liệu được dự đoán là giao dịch bất thường (True).
- **to_csv(..., encoding='utf-8-sig')**: Xuất toàn bộ danh sách giao dịch gian lận ra file CSV mới. Chuẩn mã hóa utf-8-sig giúp file CSV hiển thị chính xác các ký tự tiếng Việt và ký tự đặc biệt khi mở bằng Microsoft Excel.

```
print(f"Tổng số giao dịch phân tích: {len(df)}")
print(f"Phát hiện: {len(anomalies)} giao dịch bất thường.")
print(f"-> Đã xuất ra file thành công: '{output_filename}'!")
print(f"Phát hiện {len(anomalies)} giao dịch nghi ngờ bất thường trên tổng số {len(df)} giao dịch:")
print(anomalies[['amount_usd', 'card_type', 'channel', 'device_type', 'is_anomaly']].head(10))
```

- **print(...)**: In thống kê tổng quan ra màn hình Terminal/VS Code để kiểm tra:
 - Tổng số bản ghi dữ liệu đã quét.
 - Số lượng giao dịch bị nghi ngờ gian lận.
 - In xem trước (head(10)) 10 dòng giao dịch gian lận đầu tiên cùng với một số thông tin quan trọng như số tiền, loại thẻ, kênh và thiết bị giao dịch.

6. Phân chia tập dữ liệu và Đánh giá hiệu năng mô hình (Evaluation Metrics)

Nhằm đánh giá chính xác độ hiệu quả của mô hình Isolation Forest theo đúng tiêu chí thực nghiệm trong học máy, chương trình tiến hành chia tập dữ liệu huấn luyện – kiểm thử độc lập và tính toán các chỉ số đo lường.

A. Khai báo các hàm đo lường và Chia tập dữ liệu (Train / Test Split)

```
# Thêm các thư viện chia tập dữ liệu & đánh giá chỉ số theo yêu cầu
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, classification_report
```

```
# Đổi cột nhãn thực tế 'is_fraud' về dạng nhị phân 1/0 nếu đang ở dạng True/False hoặc 1/0
y_true = df['is_fraud'].astype(int)

# Chia dữ liệu theo tỷ lệ 80% Train, 20% Test
X_train, X_test, y_train, y_test = train_test_split(df[all_features], y_true, test_size=0.2, random_state=42)
```

- **train_test_split(...)**: Chia toàn bộ tập dữ liệu thành 2 phần tách biệt:
 - **Tập Huấn luyện (X_train)**: Chiếm **80%** dữ liệu, dùng để mô hình học cấu trúc và

hình thái các giao dịch.

- **Tập Kiểm thử (X_test):** Chiếm **20%** dữ liệu (test_size=0.2), giữ nguyên hoàn toàn độc lập để đánh giá khả năng phát hiện gian lận thực tế.
- **y_true:** Lấy nhãn thực tế gian lận (is_fraud) quy đổi sang dạng số 1 (Gian lận) và 0 (Bình thường) để làm đáp án đối chiếu.
- **random_state=42:** Cố định cách chia dữ liệu ngẫu nhiên để mọi lần chạy lại đều cho ra kết quả nhất quán.

B. Tiền xử lý độc lập và Dự đoán trên tập Kiểm thử

```
# Fit preprocessor trên tập Train và transform tập Test
X_train_enc = preprocessor.fit_transform(X_train)
X_test_enc = preprocessor.transform(X_test)

# Fit lại mô hình trên tập Train và dự đoán tập Test
model_eval = IsolationForest(contamination=0.015, n_estimators=100, random_state=42, n_jobs=-1)
model_eval.fit(X_train_enc)

preds_test = model_eval.predict(X_test_enc)
# Quy đổi dự đoán của Isolation Forest (-1: Bất thường -> 1, 1: Bình thường -> 0)
y_pred_test = np.where(preds_test == -1, 1, 0)
```

preprocessor.fit_transform(X_train) & transform(X_test): Bộ tiền xử lý chỉ học quy mô chuẩn hóa trên tập Train, sau đó áp dụng sang tập Test. Việc này giúp tuân thủ nguyên tắc **chống rò rỉ dữ liệu (Data Leakage)** trong Machine Learning.

np.where(preds_test == -1, 1, 0): Quy đổi kết quả dự đoán kỹ thuật của Isolation Forest (-1 là bất thường, 1 là bình thường) thành 1 và 0 để khớp với cột đáp án y_test

C. Tính toán và In các Chỉ số Đánh giá (Metrics)

```
# In kết quả đánh giá thực nghiệm lên Terminal
print("\n=== CÁC CHỈ SỐ ĐÁNH GIÁ MÔ HÌNH (EVALUATION METRICS) ===")
print(f"1. Accuracy (Độ chính xác): {accuracy_score(y_test, y_pred_test):.4f}")
print(f"2. Precision (Độ chuẩn xác): {precision_score(y_test, y_pred_test, zero_division=0):.4f}")
print(f"3. Recall (Độ nhạy): {recall_score(y_test, y_pred_test, zero_division=0):.4f}")
print(f"4. F1-Score (Điểm F1): {f1_score(y_test, y_pred_test, zero_division=0):.4f}")
print("\n--- CHI TIẾT BÁO CÁO PHÂN LOẠI (CLASSIFICATION REPORT) ---")
print(classification_report(y_test, y_pred_test, target_names=['Bình thường (0)', 'Bất thường (1)'], zero_division=0))
```

accuracy_score(...): Tính tỷ lệ dự đoán đúng trên toàn bộ tập kiểm thử (bao gồm cả giao dịch bình thường và bất thường).

precision_score(...): Đo lường **Độ chuẩn xác**. Cho biết trong tất cả các giao dịch mà mô hình gán cờ cảnh báo bất thường, có bao nhiêu % thực sự là giao dịch gian lận (đánh giá mức độ hạn chế báo động giả).

recall_score(...): Đo lường **Độ nhạy**. Cho biết mô hình tóm được bao nhiêu % giao dịch gian lận trên tổng số các vụ gian lận thực sự xảy ra (chỉ số quan trọng nhất đối với hệ thống tài chính ngân hàng để tránh bỏ sót tội phạm).

f1_score(...): Chỉ số trung bình hài hòa giữa Precision và Recall, phản ánh năng lực tổng thể của mô hình khi xử lý tập dữ liệu bị mất cân bằng (Imbalanced Data).

classification_report(...): In ra báo cáo tổng hợp chi tiết tất cả các chỉ số theo từng nhóm dữ liệu